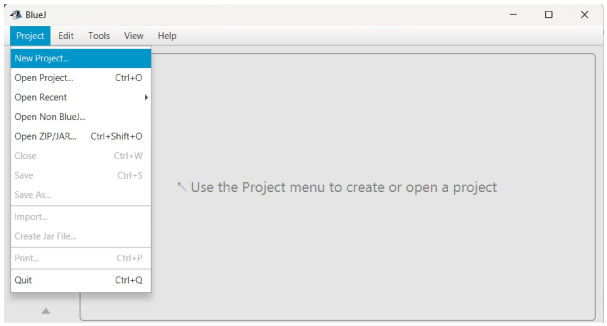
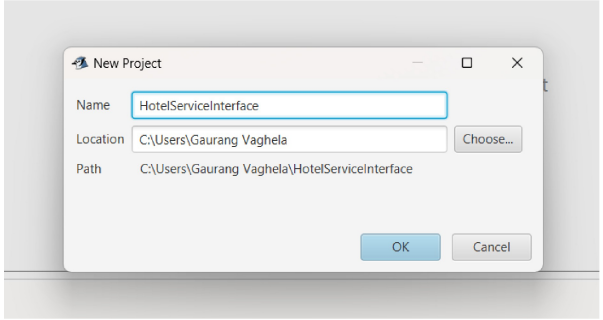
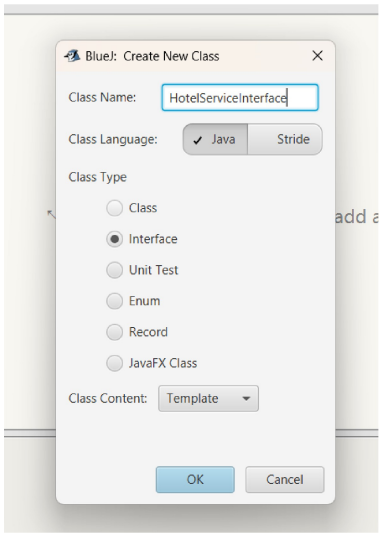
Name: Gaurang Vaghela
Rollno: BEAD-22561
DC Practical 9
Problem Statement: Design and develop a distributed Hotel booking application using Java RMI. A distributed hotel booking system consists of the hotel server and the client machines. The server manages hotel rooms booking information. A customer can invoke the following operations at his machine.
i) Book the room for the specific guest
ii) Cancel the booking of a guest.

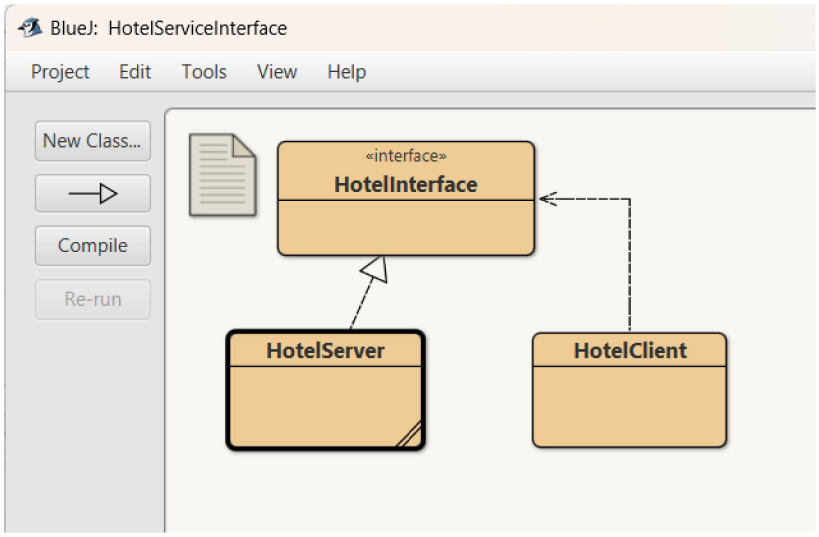
1. **Create Project**





2. **Create Interface – HotelServiceInterface and Class – HotelServer and HotelClient**





3. **Write code in the Classes**

HotelInterface.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface HotelInterface extends Remote {
        String bookRoom(String guestName) throws RemoteException;
        String cancelBooking(String guestName) throws RemoteException;
}
```

HotelServer.java

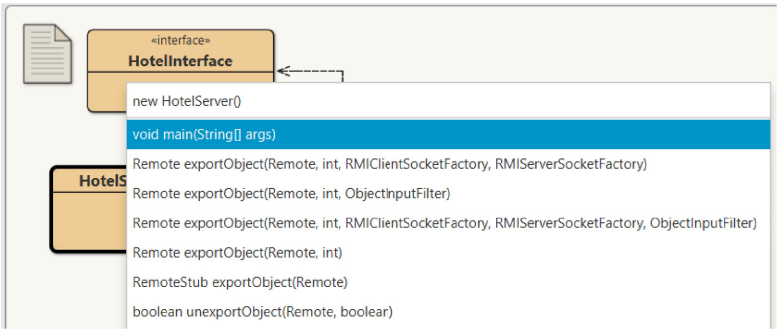
```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.HashMap;

public class HotelServer extends UnicastRemoteObject implements HotelInterface {

    HashMap<String, Boolean> bookings;

    protected HotelServer() throws RemoteException {
        bookings = new HashMap<>();
    }

    public String bookRoom(String guestName) throws RemoteException {
        if (bookings.containsKey(guestName)) {
            return "Room already booked for " + guestName;
        }
        bookings.put(guestName, true);
        return "Room booked successfully for " + guestName;
    }

    public String cancelBooking(String guestName) throws RemoteException {
        if (!bookings.containsKey(guestName)) {
            return "No booking found for " + guestName;
        }
        bookings.remove(guestName);
        return "Booking cancelled for " + guestName;
    }

    public static void main(String[] args) {
        try {
            HotelServer server = new HotelServer();

            Registry registry = LocateRegistry.createRegistry(1099);
            registry.rebind("HotelService", server);

            System.out.println("Hotel Server is running...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

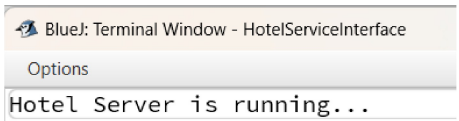
HotelClient.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class HotelClient {

    public static void main(String[] args) {
```

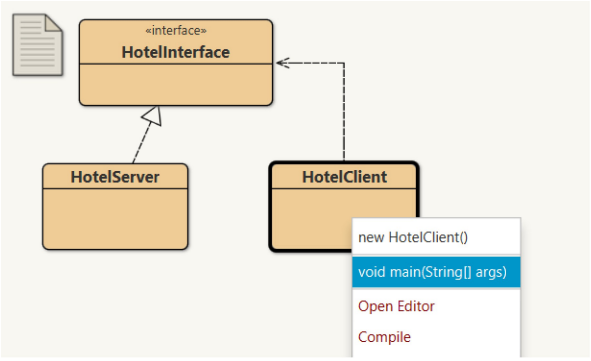
```
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            HotelInterface hotel = (HotelInterface) registry.lookup("HotelService");

            Scanner sc = new Scanner(System.in);

            while (true) {
                System.out.println("\n1. Book Room");
                System.out.println("2. Cancel Booking");
                System.out.println("3. Exit");
                System.out.print("Enter choice: ");

                int choice = sc.nextInt();
                sc.nextLine();

                if (choice == 1) {
                    System.out.print("Enter guest name: ");
                    String name = sc.nextLine();
                    System.out.println(hotel.bookRoom(name));

                } else if (choice == 2) {
                    System.out.print("Enter guest name: ");
                    String name = sc.nextLine();
                    System.out.println(hotel.cancelBooking(name));

                } else {
                    break;
                }
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

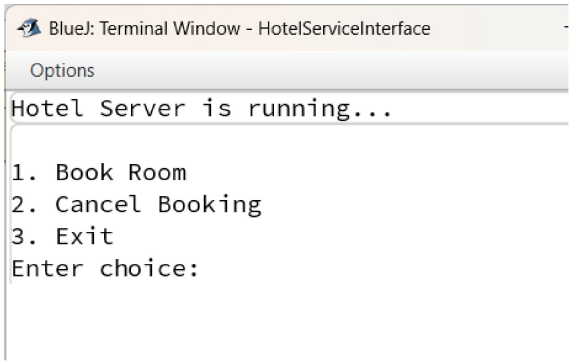
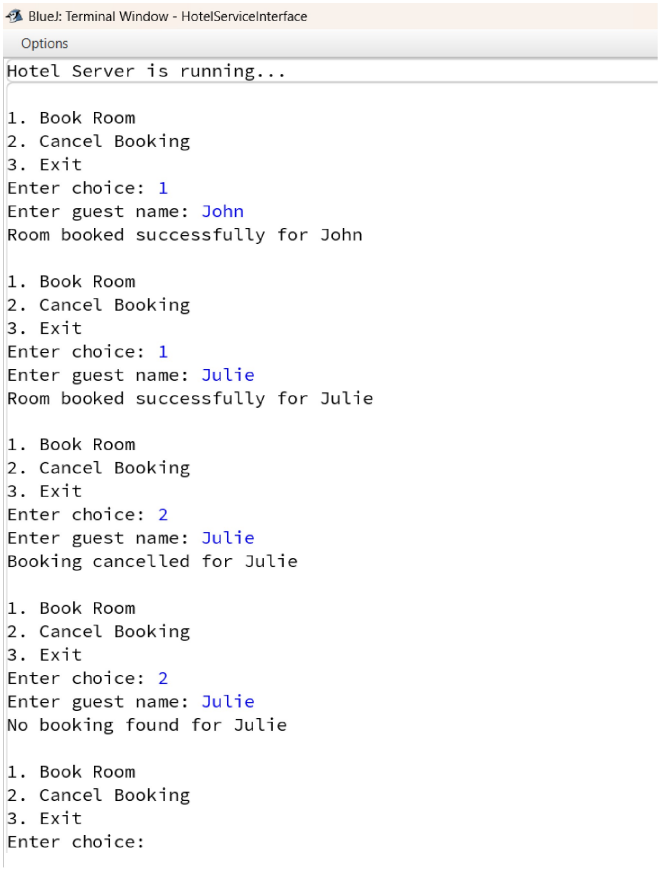
4. Run main() function of HotelServer





5. Run main() function of HotelClient





6. Provide appropriate Input